A Project Report on

MY PERSONAL CHEF

A Project Thesis is Submitted to The Jawaharlal Nehru Technological University, Kakinada. In Partial Fulfilment of the Requirements for the Award of the Degree of

BACHELOR OF TECHNOLOGY

In

COMPUTER SCIENCE AND ENGINEERING

Submitted By

D.LAKSHMAN	22NF1A0511
A.YESWANTH SAI RAM	22NF1A0502
G.KALPANA	22NF1A0513
CH.KARTHIK	22NF1A0507
M.LATHASRI	22NF1A0547

Under the Guidance of

Mr. D.V.V.BRAMHACHARI , MCA, M.Tech

Assistant Professor

DEPARTMENT OF COMPUTERSCINENCE AND ENGINEERING

UNIVERSAL COLLEGE OF ENGINEERING AND TECHNOLOGY AUTONOMOUS

Affiliated to JNTUK, Kakinada& Approved by AICTE Delhi, Accreted by NAAC B++

DOKIPARRU (V), MEDIKONDURU(M), GUNTUR(Dt.)-522438, A.P, 2025-26

UNIVERSAL COLLEGE OF ENGINEERING AND TECHNOLOGY AUTONOMOUS

(Approved by A.I.C.T.E. Accreted by NAACB++, Affiliated to J.N.T.U.K, Kakinada)

DOKIPARRU (V), MEDIKONDURU(M), GUNTUR(Dt.)-522438 Andhra Pradesh

DEPARTMENT OF COMPUTER SCINENCE AND ENGINEERING

CERTIFICATE

This is to certify that the Project report entitled **"MY PERSONAL CHEF"** is being submitted by D.LAKSHMAN(22NF1A0511), A.YESWANTH SAI RAM (22NF1A0502),G.KALPANA(22NF1A0513),CH.KARTHIK(22NF1A0507),M.LATHA SRI(22NF1A0547) in the department of **COMPUTER SCIENCE AND ENGINEERING, Universal College of Engineering and Technology,** Dokiparru, Guntur is a record of Bonafide work carried out in the Year 2025 in partial fulfilment of the requirements for the award of degree of **BACHELOR OF TECHNOLOGY** in **COMPUTER SCIENCE AND ENGINERRING** under my guidance and supervision.

Internal Guide	**Head of the department**
Mr.D.V.V.BRAMHACHARI	**Dr. M.VEERA KUMARI**

External Examiner

ACKNOWLEDGEMENT

We feel immense pleasure to express our sincere thanks and profound sense of gratitude to all those people who played a valuable role in the successful completion of our project with their invaluable suggestions and advice.

We are very thankful for the completion of our project to our secretary & correspondent of our college **Rev.Fr.Dr.P.BALA SWAMY,** for his encouragement and motivation to complete my main project.

We are thankful to our principal **Dr.M.KOTI REDDY,** Ph.D, for permitting and encouraging us in doing this project.

We are deeply intended **to Dr.M.VEERA KUMARI**, M.Tech, Ph.D,Associate Professor and Head of the Department of Computer science & Engineering, whose motivation and constant encouragement have led to the pleasure of a project in the field of software development.

We are very obligated and thankful to my internal guide **D.V.V.BRAMHACHARI,** Assistant Professor and department of Computer science & Engineering for providing opportunity and constant encouragement given by him during the course. We are grateful for his valuable guidance and suggestions during our main project.

We would like to extend my deepest gratitude to my parents who always help me with best of the facilities for our bright future.

Finally, We thank to each person who directly and indirectly contributed his or her efforts to complete this project.

Project Associates

D. LAKSHMAN	22NF1A0511
A. YESWANTH SAI RAM	22NF1A0502
G. KALPANA	22NF1A0513
CH. KARTHIK	22NF1A0507
M. LATHASRI	22NF1A0547

DECLARATION

We hereby declare that the project entitled **"My Personal Chef"** submitted to **Jawaharlal Nehru Technological University, Kakinada**, in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering**, is a record of original work carried out by us.

We further declare that this project has not been submitted elsewhere for the award of any degree or diploma.

Name of the Student	Register Number	Signature
D. LAKSHMAN	22NF1A0511	
A. YESWANTH SAI RAM	22NF1A0502	
G. KALPANA	22NF1A0513	
CH. KARTHIK	22NF1A0507	
M. LATHASRI	22NF1A0547	

ABSTRACT

The **My Personal Chef** system is a web-based application designed to help users discover recipes and receive personalized cooking suggestions based on available ingredients. The system is developed using the **MERN stack**, which includes **MongoDB, Express.js, React.js, and Node.js**.

The application allows users to browse recipes, search for dishes, view detailed cooking instructions, and save favourite recipes. One of the key features of the system is the **Pantry Matching feature**, which suggests recipes based on ingredients entered by the user.

The system also includes an **administrative module** where administrators can add, update, and manage recipes. Additionally, the system integrates **Artificial Intelligence using Google Gemini AI** to automatically extract recipe details from YouTube cooking videos.

The application provides an interactive and user-friendly interface that simplifies recipe discovery and enhances the cooking experience for users.

CONTENTS

Sl. No	Description	Page No
1	**INTRODUCTION**	**1 – 3**
2	**SYSTEM ANALYSIS**	**4 – 9**
2.1	Existing System	5
2.2	Proposed System	5
2.3	Module Description	6
2.4	Hardware Requirements	7
2.5	Software Requirements	7
2.6	Feasibility Study	8
2.6.1	Technical Feasibility	8
2.6.2	Operational Feasibility	8
2.6.3	Economic Feasibility	8
2.7	Functional Requirements	9
3	**SYSTEM DESIGN**	**10 – 23**
3.1	System Architecture	10
3.2	UML Diagrams	13
3.2.1	Class Diagram	14
3.2.2	Use Case Diagram	15
3.2.3	Activity Diagram	16
3.2.4	Sequence Diagram	17
3.3	Technologies Description	18 – 23
4	**CODING AND IMPLEMENTATION**	**24 – 32**
5	**OUTPUT SCREENS**	**33 – 41**
6	**TESTING**	**42 – 47**
6.1	Testing Technologies	44
6.2	Test Cases	45
7	**CONCLUSION**	**48 - 49**
8	**FUTURE ENHANCEMENTS**	**50 - 51**
9	**BIBLIOGRAPHY**	**52 - 54**
10	**JOURNAL PUBLICATION**	**55**

LIST OF FIGURES

Fig No	Description	Page No
Fig 3.2.1	Class Diagram of My Personal Chef System	14
Fig 3.2.2	Use Case Diagram	15
Fig 3.2.3	Activity Diagram	16
Fig 3.2.4	Sequence Diagram	17
Fig 5.1	Home Page of My Personal Chef Application	34
Fig 5.2	Login Page	35
Fig 5.3	Signup Page	36
Fig 5.4	Recipes Page	37
Fig 5.5	Recipe Detail Page	38
Fig 5.6	Pantry Suggestion Page	39
Fig 5.7	Favourites Page	40
Fig 5.8	Admin Add Recipe Page	41
Fig 5.9	Admin Recipe Management Page	42
Fig 5.10	Admin Feedback Management Page	43

CHAPTER – 1

INTRODUCTION

Introduction

In today's digital world, the use of online platforms for everyday activities has increased significantly. One such activity is cooking and meal planning. Many people rely on the internet to discover new recipes, learn cooking techniques, and explore different cuisines from around the world. Although a large number of recipe websites and cooking applications are available, users often face difficulties when searching for suitable recipes. Recipes are usually scattered across multiple platforms, and users have to spend considerable time browsing different websites to find dishes that match their preferences and available ingredients.

Another common challenge faced by users is deciding what to cook using the ingredients they already have at home. Many recipe platforms provide large collections of recipes, but they do not provide efficient methods to recommend recipes based on the ingredients available with the user. As a result, users may struggle to find appropriate recipes and may even waste ingredients that could otherwise be used effectively. Therefore, there is a need for a system that can simplify recipe discovery and provide intelligent suggestions based on available ingredients.

The project **"My Personal Chef"** is developed to address these problems by creating a centralized web-based recipe management and recommendation system. The system provides an organized platform where users can browse, search, and manage cooking recipes easily. Users can explore a collection of recipes stored in the database and view detailed cooking instructions that include ingredients, preparation steps, cooking time, and difficulty level.

One of the most important features of the system is the **Pantry Suggestion feature**. This feature allows users to enter the ingredients they currently have in their kitchen. The system then analyses these ingredients and compares them with the ingredient lists stored in the recipe database. Based on the number of matching ingredients, the system recommends recipes that the user can prepare with minimal additional ingredients. This feature helps users make better decisions when choosing recipes and also helps reduce food wastage.

The application is developed using modern web technologies that form the **MERN stack**, which includes MongoDB, Express.js, React.js, and Node.js. React.js is used to build the frontend user interface, while Node.js and Express.js handle the backend operations and business logic. MongoDB is used as the database to store recipes, user data, favourites, and feedback information. The system also integrates **Gemini AI**, which helps administrators

automatically extract recipe information from YouTube cooking videos, reducing manual effort and improving the efficiency of adding new recipes.

In addition, the system is designed with a user-friendly interface that allows users to interact with the platform easily and efficiently. The structured layout of recipes, search functionality, and filtering options make it simple for users to explore different dishes according to their preferences. By combining modern web technologies with intelligent recommendation features, the **My Personal Chef** system provides an effective solution for digital recipe management and improves the overall experience of discovering and preparing meals.

Overall, the **My Personal Chef** system provides a convenient platform for discovering recipes and planning meals efficiently. By combining modern web technologies with intelligent ingredient-based recommendations, the system improves the user experience and demonstrates the practical implementation of a full-stack web application.

CHAPTER – 2

SYSTEM ANALYSIS

2. SYSTEM ANALYSIS

2.1 Existing System

The existing system for recipe discovery mainly relies on searching through multiple online platforms such as recipe websites, cooking blogs, and YouTube videos. Although these platforms provide a large number of recipes, they often lack proper organization and personalized recommendations.

Users usually have to manually search for recipes based on their preferences or available ingredients. Many platforms do not provide suggestions based on ingredients already available in the user's kitchen. In addition, several websites contain excessive advertisements and unstructured content, which makes finding suitable recipes time-consuming.

Because of these limitations, existing systems lack efficiency, personalization, and intelligent recommendation features.

2.2 PROPOSED SYSTEM

The proposed system **"My Personal Chef"** is a web-based recipe management and recommendation platform that helps users easily discover and manage recipes. Users can browse recipes, search for dishes, view cooking instructions, and receive suggestions based on the ingredients available in their kitchen.

The application is developed using **React.js, Node.js, Express.js, and MongoDB**, forming the MERN stack architecture. The system also includes a **Pantry Suggestion feature**, which recommends recipes based on ingredients entered by the user.

In addition, an **admin module** allows administrators to add, update, and delete recipes and manage feedback. The system also integrates **Gemini AI** to extract recipe information from YouTube videos. By providing intelligent suggestions and centralized recipe management, the system improves the overall cooking experience for users.

2.3 MODULE DESCRIPTION

Module 1: User Management Module

- Allows users to register and create an account.
- Provides login functionality for users.
- Allows users to manage their profile information.
- Handles user authentication and authorization.

Module 2: Recipe Management Module

- Allows users to browse recipes stored in the database.
- Provides search functionality for finding recipes by title.
- Allows filtering recipes by category and type.
- Displays detailed cooking instructions including ingredients and preparation steps.

Module 3: Favourites Module

- Allows users to save recipes as favourites.
- Enables users to view and manage their saved recipes.
- Provides quick access to frequently used recipes.

Module 4: Pantry Suggestion Module

- Allows users to enter ingredients they have available.
- Compares user ingredients with recipe ingredient lists.
- Suggests recipes with the highest number of matching ingredients.
- Displays ranked recipe suggestions.

Module 5: Admin Module

- Allows administrators to add new recipes.
- Allows administrators to edit and update recipe information.
- Enables deletion of recipes from the database.

- Allows moderation of user feedback.

Module 6: Database Module

- Stores user data, recipes, favourites, and feedback.
- Provides data retrieval and storage functionality.
- Ensures efficient data management and integrity.

2.4 HARDWARE REQUIREMENTS

System : Minimum Intel i3 Processor or above

Hard Disk : 40 GB (Minimum)

RAM : 4 GB (Minimum)

2.5 SOFTWARE REQUIREMENTS

Operating System : Windows 8 and above

Frontend : React.js

Backend : Node.js, Express.js

Database : MongoDB

Styling Framework : Tailwind CSS

Development Tool : Visual Studio Code

API Testing Tool : Thunder Client

2.6 FEASIBILITY STUDY

A feasibility study evaluates whether the proposed system is practical and achievable before implementation. It examines the technical, operational, and economic aspects of developing the **My Personal Chef** system.

2.6.1 Technical Feasibility

Technical feasibility evaluates whether the required technologies and tools are available to develop and maintain the system. The My Personal Chef system uses modern and widely supported technologies such as React.js, Node.js, Express.js, and MongoDB. These technologies are open source and suitable for developing scalable web applications. Therefore, the system is technically feasible to implement.

2.6.2 Operational Feasibility

Operational feasibility examines whether the system can be successfully used by its intended users. The My Personal Chef application is designed with a simple and user-friendly interface, allowing users to easily browse recipes, search for dishes, and receive ingredient-based suggestions. Administrators can also manage recipe data efficiently. Hence, the system is operationally feasible.

2.6.3 Economic Feasibility

Economic feasibility evaluates the cost-effectiveness of the project. The system uses open-source technologies and free development tools, which significantly reduces development costs. Since no expensive software or hardware is required, the project is economically feasible.

2.7 FUNCTIONAL REQUIREMENTS

User Management

1. User Registration: Users can create an account.
2. User Login: Users can log in to access system features.
3. Profile Management: Users can update their profile information.

Recipe Management

1. Recipe Browsing: Users can browse available recipes.
2. Recipe Search: Users can search recipes by title.
3. Recipe Filtering: Users can filter recipes by category and type.

Favorites Management

1. Save Favorites: Users can save recipes as favorites.
2. View Favorites: Users can access their saved recipes.

Pantry Suggestion System

1. Ingredient Input: Users can enter available ingredients.
2. Recipe Matching: System matches ingredients with stored recipes.
3. Recipe Recommendation: System suggests recipes based on matching ingredients.

Admin Control

1. Recipe Management: Admin can add, edit, and delete recipes.
2. Feedback Management: Admin can review and moderate user feedback.

Security and Access Control

1. User Authentication: System verifies user credentials.
2. Role-Based Access Control: Admin and users have different access levels.
3. Data Security: User and recipe data are stored securely in the database.

CHAPTER – 3

SYSTEM DESIGN

3. SYSTEM DESIGN

3.1 System Architecture

3.1.1. Frontend (Client Interface)

Technologies: React.js, HTML, CSS, JavaScript, Tailwind CSS

Functionality:
The frontend of the **My Personal Chef** system provides the user interface through which users interact with the application. It allows users to browse recipes, search recipes by title, and filter recipes by category and type. The interface also displays detailed recipe information such as ingredients, cooking steps, cooking time, and difficulty level.

Users can save recipes to their favourites and access them later. The pantry feature allows users to enter the ingredients they currently have, and the system suggests recipes based on ingredient matches. The frontend communicates with the backend server using API requests to retrieve and update data.

3.1.2. Backend (Application Logic)

Framework: Node.js and Express.js

Functionality:
The backend handles the core application logic of the system. It processes requests from the frontend and performs operations such as retrieving recipes, adding new recipes, updating recipe information, and deleting recipes.

The backend also manages user authentication, favourites management, pantry suggestion processing, and feedback handling. It communicates with the MongoDB database using Mongoose and ensures efficient data storage and retrieval.

3.1.3. AI Integration Module

Technology: Gemini AI (Google Generative AI)

Functionality:
The AI module helps administrators automatically extract recipe information from YouTube cooking videos. When an admin enters a YouTube URL, the system analyses the video transcript or description using Gemini AI and extracts relevant recipe details such as title, ingredients, steps, cooking time, and images.

This feature reduces manual effort when adding recipes and improves the efficiency of maintaining the recipe database.

3.1.4. Database

Type: MongoDB

Usage:
MongoDB is used as the database for storing application data. The database stores collections such as user accounts, recipes, favorites, and feedback. Each recipe document contains fields like title, ingredients, cooking steps, image, cooking time, difficulty level, and category. The database allows efficient storage, retrieval, and management of data used in the system.

3.1.5. Media / Static Resources

The system includes static resources used for the visual interface and user interaction.

Resources include:

- Recipe images
- Icons and UI components
- CSS files for styling
- JavaScript files for interactivity

These resources help create a visually appealing and user-friendly interface.

3.2 UML DIAGRAMS

UML stands for **Unified Modeling Language**. It is a standardized modeling language used in software engineering to visualize, specify, construct, and document the components of a software system. UML diagrams help developers understand the structure and behavior of the system before implementation.

UML provides graphical representations of system components and their relationships, making it easier to analyze and design complex software systems.

Goals of UML

1. Provide a visual modeling language to represent system design.
2. Allow developers to understand and communicate system architecture.
3. Be independent of programming languages and development processes.
4. Provide a formal foundation for system modeling.
5. Encourage better software design practices.
6. Support high-level concepts such as components and system architecture.
7. Improve documentation and maintainability of software systems.

3.2.1 CLASS DIAGRAM

The class diagram represents the structure of the **My Personal Chef** system. It shows the classes used in the system and their relationships. Important classes include **User, Recipe, Favorites, and Feedback**.

The **Recipe class** contains information such as recipe title, ingredients, steps, cooking time, and difficulty level. The **User class** stores user information such as email and password. The **Favorites class** manages the recipes saved by users, while the **Feedback class** stores user feedback related to recipes.

The class diagram helps organize the system structure and ensures proper relationships between different components of the application.

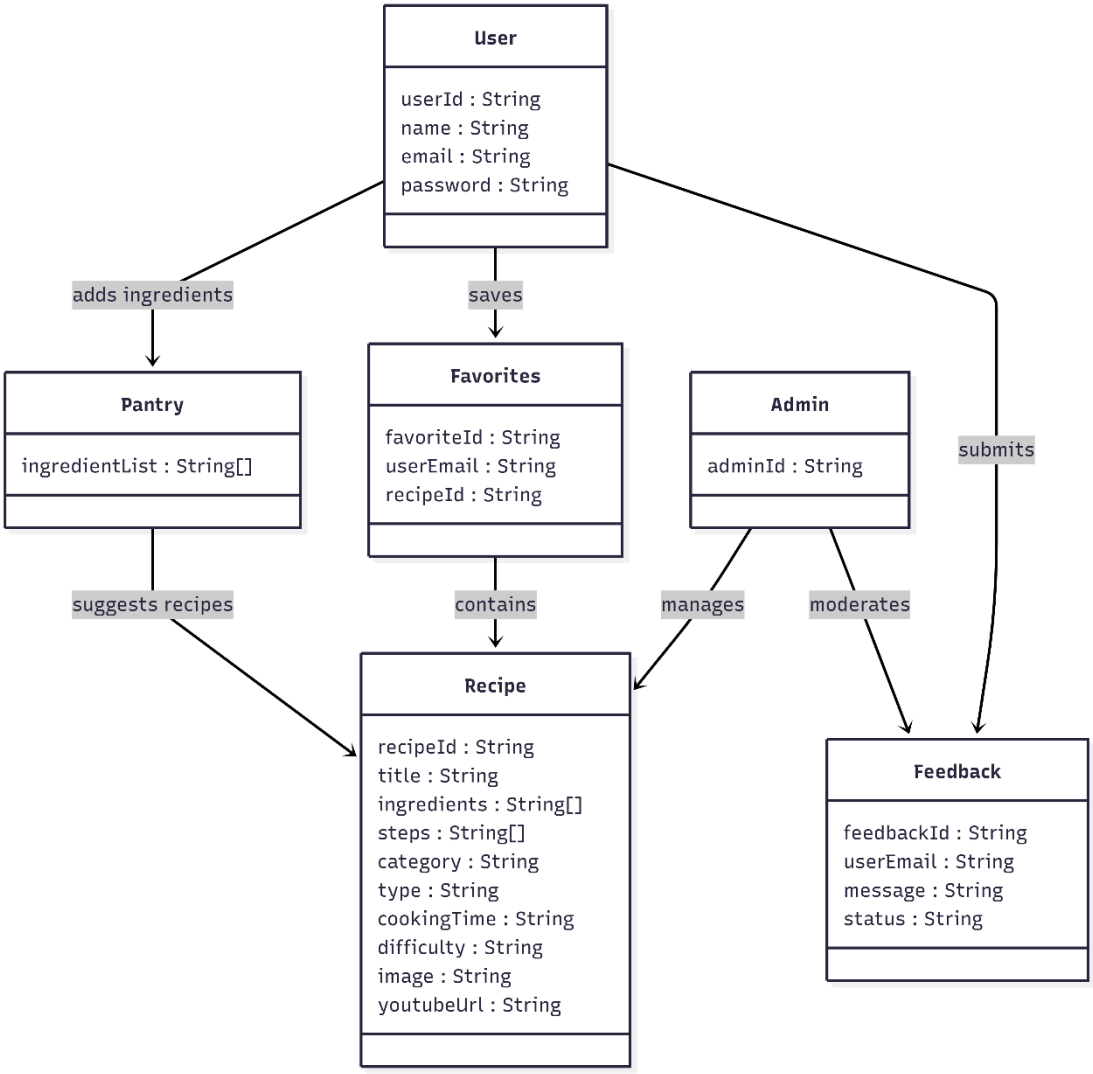


Fig 3.2.1: Class Diagram

3.2.2 USE CASE DIAGRAM

The use case diagram represents how different users interact with the system. The main actors in the system are **User** and **Administrator**.

Users can browse recipes, search recipes, view recipe details, add recipes to favorites, and receive pantry-based recipe suggestions. Administrators can add new recipes, update existing recipes, delete recipes, and manage user feedback.

The use case diagram helps understand the interaction between users and the system functionalities.

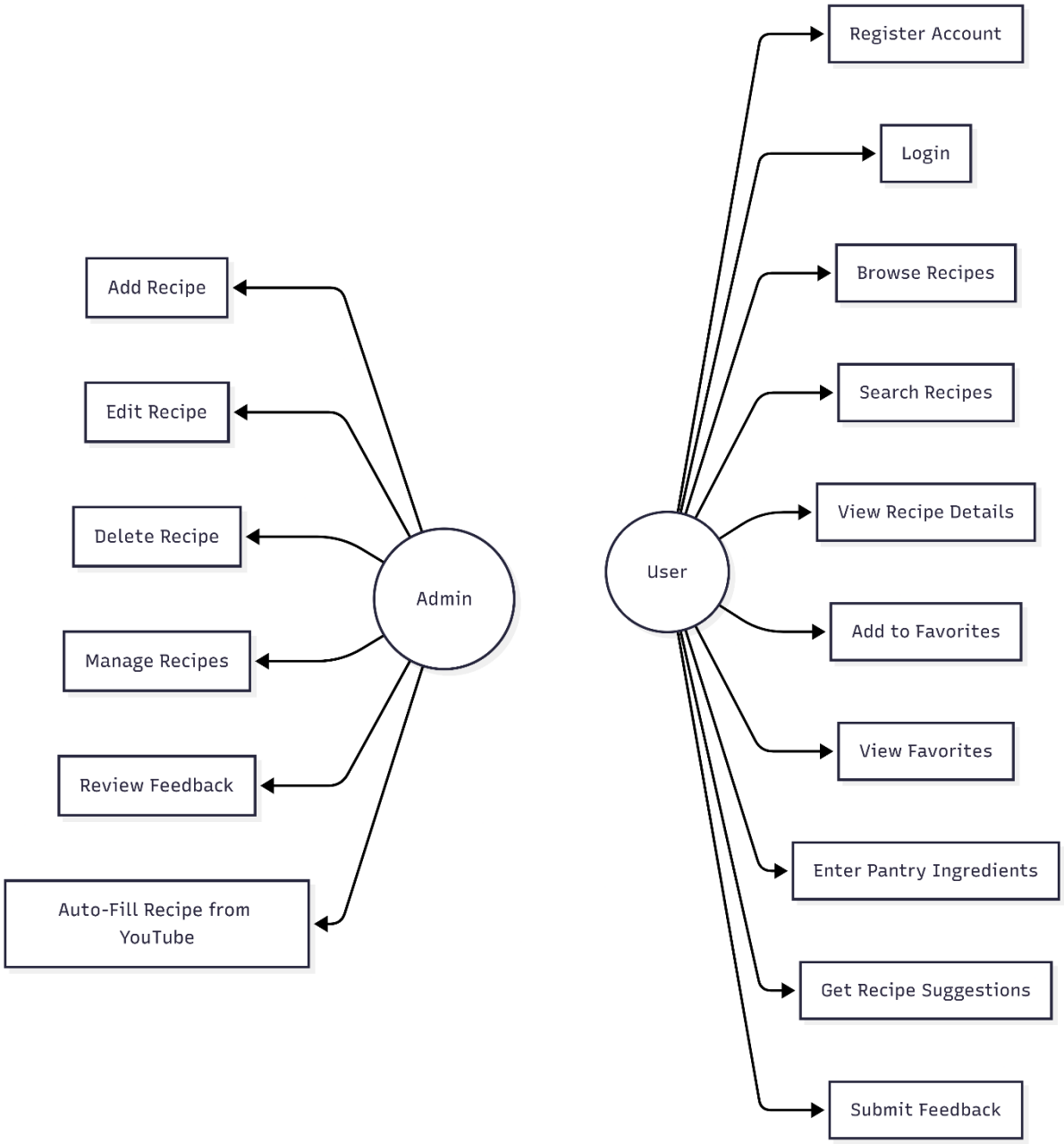


Fig 3.2.2: Use Case Diagram

3.2.3 ACTIVITY DIAGRAM

The activity diagram represents the workflow of the **My Personal Chef** system. It shows the sequence of steps involved when users interact with the application.

For example, when a user enters ingredients in the pantry feature, the system processes the input, compares the ingredients with recipe data, and displays matching recipes. The activity diagram helps visualize the flow of operations within the system.

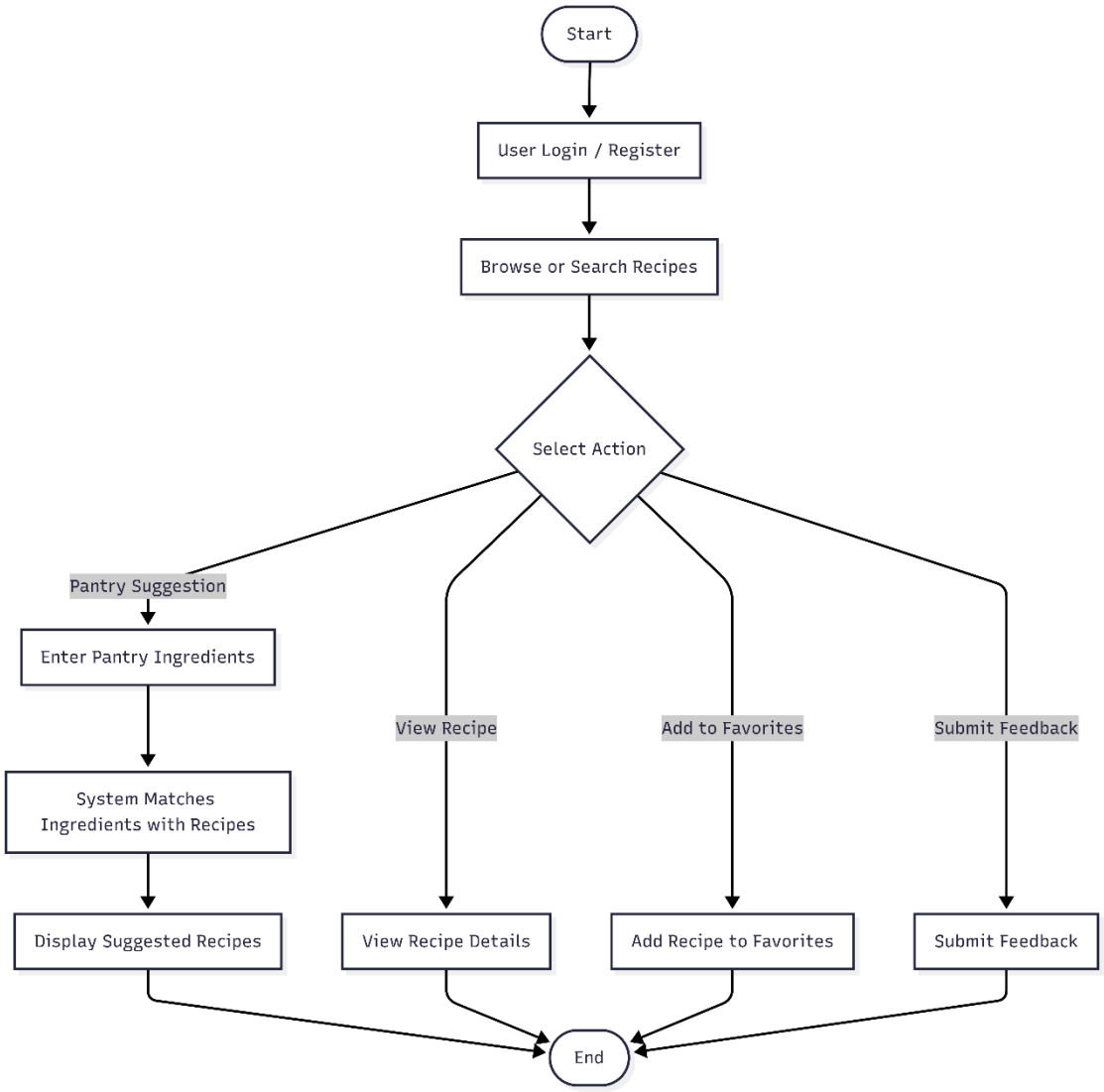


Fig 3.2.3: Activity Diagram

3.2.4 SEQUENCE DIAGRAM

The sequence diagram illustrates the interaction between system components over time. It shows how the user interface, backend server, AI module, and database communicate with each other.

For example, when a user searches for a recipe, the request is sent from the frontend to the backend server. The backend retrieves the data from the MongoDB database and sends the results back to the frontend for display.

The sequence diagram helps understand how system components interact during different operations.

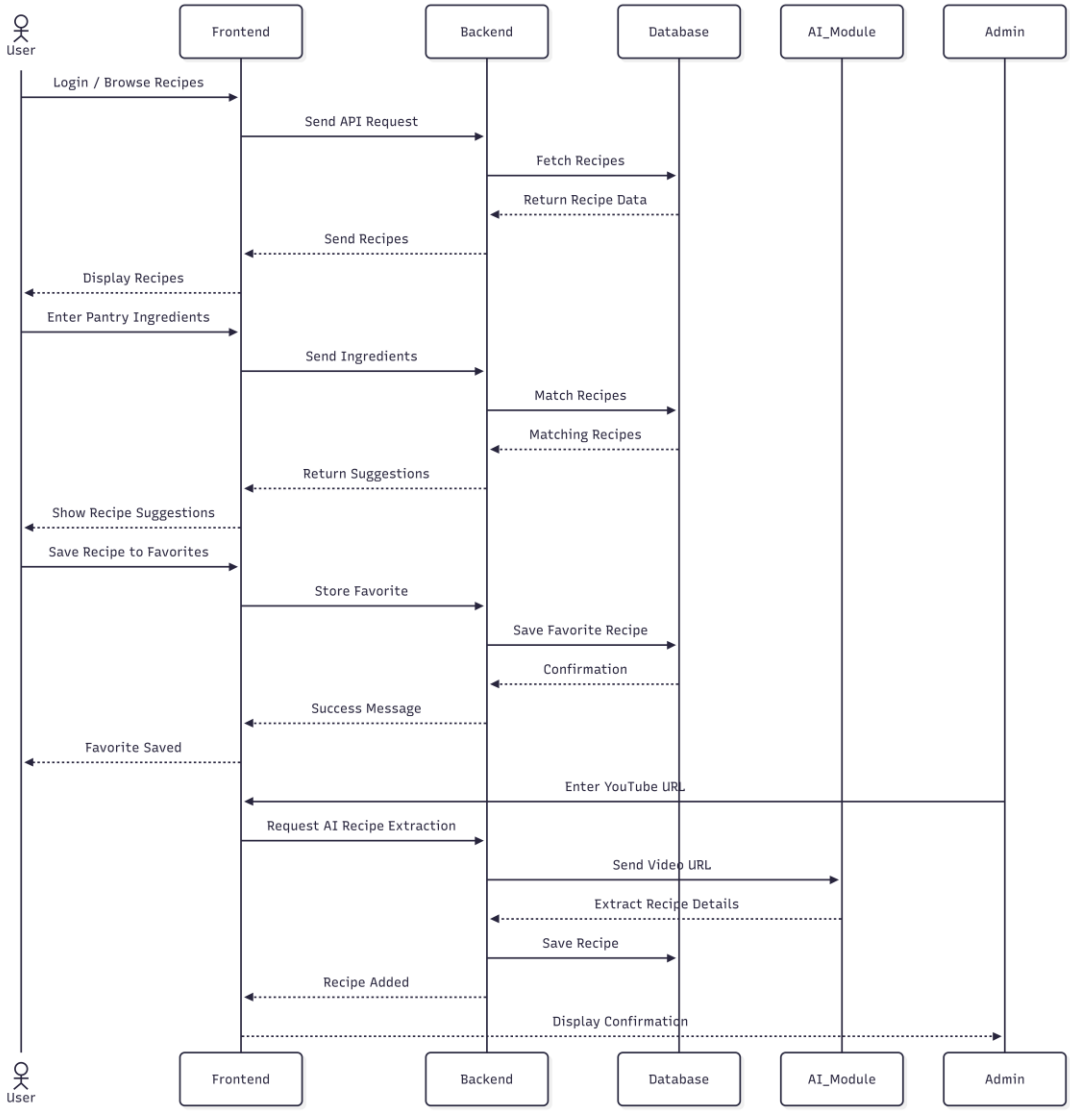


Fig 3.2.4: Sequence Diagram

3.3 TECHNOLOGIES DESCRIPTION

3.3.1 React.js

React.js is a popular JavaScript library used for building modern user interfaces, especially for single-page applications. It was developed and maintained by Facebook and is widely used for creating fast and interactive web applications. React allows developers to build reusable UI components that can efficiently update when data changes.

One of the key advantages of React is its **component-based architecture**, where the user interface is divided into independent and reusable components. Each component manages its own logic and rendering, making the application easier to maintain and scale.

React uses a concept known as the **Virtual DOM (Document Object Model)**. Instead of updating the entire web page whenever a change occurs, React updates only the specific components that need modification. This improves performance and provides a smoother user experience.

In the **My Personal Chef** application, React is used to build the frontend interface where users can browse recipes, search for dishes, view cooking instructions, and manage their favorite recipes. It also handles navigation between different pages such as the home page, recipe list, recipe details, favorites page, and pantry suggestions page.

React works seamlessly with other libraries such as **React Router**, which enables navigation between different pages without refreshing the entire website.

Advantages of React.js:

- Component-based architecture improves code reusability
- Fast rendering using Virtual DOM
- Strong community support
- Easy integration with backend APIs
- Efficient handling of dynamic content

3.3.2 Node.js

Node.js is an open-source runtime environment that allows JavaScript to run on the server side. It is built on the Chrome V8 JavaScript engine and is designed for building fast and scalable network applications.

Traditionally, JavaScript was used only for frontend development within web browsers. Node.js allows developers to use JavaScript for backend development as well, enabling the creation of full-stack applications using a single programming language.

Node.js follows a **non-blocking and event-driven architecture**, which makes it highly efficient for handling multiple user requests simultaneously. This architecture allows the server to process many operations at the same time without slowing down the application.

In the **My Personal Chef** project, Node.js is used to build the backend server that handles API requests from the React frontend. It processes user requests such as retrieving recipes, saving favorites, submitting feedback, and managing user accounts.

Advantages of Node.js:

- High performance due to asynchronous processing
- Scalable architecture
- Allows full-stack JavaScript development
- Large ecosystem of libraries through npm
- Suitable for real-time applications

3.3.3 Express.js

Express.js is a lightweight web framework for Node.js that simplifies the process of building web applications and APIs. It provides a robust set of features for handling HTTP requests, routing, middleware integration, and server-side logic.

Express makes it easier to create RESTful APIs that allow the frontend and backend to communicate effectively. It also supports middleware functions that help manage tasks such as authentication, request parsing, and error handling.

In the **My Personal Chef** system, Express.js is used to create REST API endpoints that allow the frontend application to interact with the database. For example, the system provides APIs to add recipes, retrieve recipe details, update recipes, delete recipes, and manage user favorites.

Express.js also helps manage different routes such as:

- /recipes
- /favorites
- /api/feedback
- /login
- /create-user

These APIs allow the application to perform various operations while maintaining a structured backend architecture.

Advantages of Express.js:

- Minimal and flexible framework
- Simplifies API development
- Easy integration with databases
- Supports middleware architecture
- High performance with Node.js

3.3.4 MongoDB

MongoDB is a NoSQL database that stores data in a flexible, JSON-like format called documents. Unlike traditional relational databases that use tables and rows, MongoDB stores data in collections and documents, making it easier to handle complex and dynamic data structures.

MongoDB is particularly well-suited for modern web applications because it allows developers to store and retrieve data quickly without strict schema limitations.

In the **My Personal Chef** application, MongoDB is used to store various types of data including:

- Recipe details
- User accounts
- Favorite recipes
- Feedback from users

Each recipe document contains fields such as:

- title
- ingredients
- steps
- cooking time
- difficulty level
- category
- type
- image
- YouTube URL

MongoDB integrates with Node.js using **Mongoose**, which provides an object data modeling (ODM) library that simplifies database operations.

Advantages of MongoDB:

- Flexible schema design
- High scalability
- Fast data retrieval
- Suitable for large datasets
- Easy integration with JavaScript applications

3.3.5 Tailwind CSS

Tailwind CSS is a utility-first CSS framework used to design modern and responsive user interfaces. Unlike traditional CSS frameworks that provide predefined components, Tailwind offers small utility classes that can be combined to create custom designs.

Tailwind CSS allows developers to apply styles directly in HTML or JSX files using predefined classes for layout, spacing, colors, typography, and responsiveness.

In the **My Personal Chef** project, Tailwind CSS is used to design the user interface of the application. It helps create a clean and responsive layout for different pages such as:

- Home page
- Recipe browsing page
- Recipe detail page
- Favorites page
- Pantry suggestion page
- Admin management pages

Tailwind also ensures that the application works well on different screen sizes including desktops, tablets, and mobile devices.

Advantages of Tailwind CSS:

- Rapid UI development
- Highly customizable
- Responsive design support
- Clean and maintainable styling
- No need for large CSS files

3.3.6 Gemini AI

Gemini AI is a generative artificial intelligence model developed by Google that is capable of understanding and generating human-like content. It can process various types of input including text, images, and structured data.

In the **My Personal Chef** application, Gemini AI is used to automatically extract recipe information from YouTube videos. When an administrator provides a YouTube video link, the system sends the video information to the Gemini API.

The AI analyzes the video description or transcript and extracts important recipe details such as:

- Recipe title
- Ingredients list
- Cooking steps
- Cooking time
- Difficulty level
- Recipe image

This information is automatically filled into the recipe form, allowing administrators to review and publish the recipe quickly.

The integration of Gemini AI significantly reduces manual work and improves the efficiency of adding new recipes to the system.

Advantages of Gemini AI:

- Automated data extraction
- Natural language understanding
- Reduces manual input effort
- Improves productivity
- Supports intelligent application features

CHAPTER-4

CODING AND IMPLEMENTATION

4. CODING AND IMPLEMENTATION

4.1 Introduction

The coding and implementation phase is the stage where the system design is converted into a functional software application. In this phase, the modules identified during system design are developed using appropriate programming languages and frameworks.

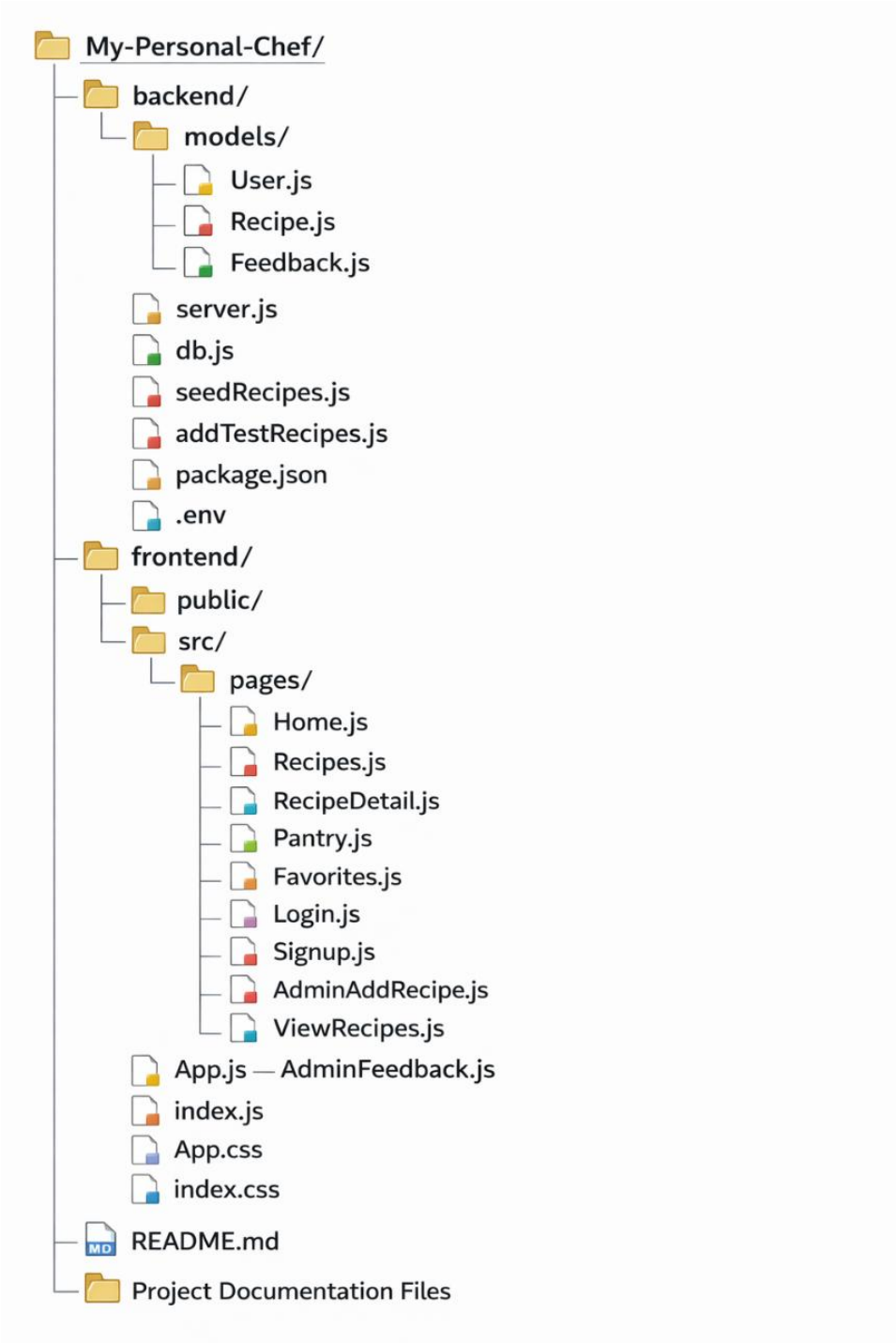
The **My Personal Chef** application is implemented using the **MERN stack**, which consists of **MongoDB, Express.js, React.js, and Node.js**. The frontend of the system is developed using **React.js**, which provides an interactive and responsive user interface for users to browse recipes, manage favorites, and get recipe suggestions based on available ingredients.

The backend is implemented using **Node.js and Express.js**, which handle API requests, authentication, database operations, and AI integration. **MongoDB** is used as the database to store information related to recipes, users, favorites, and feedback.

Additionally, the system integrates **Google Gemini AI**, which extracts recipe information from YouTube videos and automatically fills the recipe form. This feature improves efficiency and reduces manual data entry for administrators.

4.2 Project Folder Structure

The project follows a modular folder structure where frontend and backend components are organized separately. This structure improves maintainability, scalability, and readability of the application.



Folder Description

Backend Folder

The backend folder contains the server-side implementation of the system. It handles API requests, database operations, and business logic of the application.

Important files include:

- **server.js** – Main backend server file
- **db.js** – MongoDB database connection
- **models/** – Database schema definitions
- **package.json** – Backend dependencies

Frontend Folder

The frontend folder contains the React application which provides the graphical user interface for users.

Important files include:

- **App.js** – Main routing component
- **index.js** – Entry point of the React application
- **pages/** – Contains all application pages
- **CSS files** – Styling for the application

4.3 Frontend Implementation

The frontend of the system is implemented using **React.js**, which enables the development of reusable components and efficient state management.

The application uses **React Router** for navigation between pages such as Home, Recipes, Pantry, Favorites, Login, Signup, and Admin modules.

Each page in the application is implemented as a separate component. These components communicate with the backend server using **REST APIs**.

App.js

The App.js file is the main component that manages routing between pages.

Example code:

```
<Route path="/" element={<Home />} />
<Route path="/recipes" element={<Recipes />} />
<Route path="/recipe/:id" element={<RecipeDetail />} />
<Route path="/favorites" element={<Favorites />} />
<Route path="/pantry" element={<Pantry />} />
```

Login Page

The login page allows users to enter their email and password to access the system.

```
const res = await fetch("http://localhost:5000/login", {
method: "POST",
headers: {
"Content-Type": "application/json"
},
body: JSON.stringify({ email, password })
});
```

Pantry Page

The pantry page allows users to enter ingredients they already have. The system then suggests recipes based on matching ingredients.

```
const response = await fetch(
"http://localhost:5000/recipes/pantry-match",
{
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify({ ingredients })
});
```

4.4 Backend Implementation

The backend of the system is implemented using **Node.js and Express.js**. The backend server handles API requests from the frontend and performs operations such as authentication, recipe management, pantry matching, and feedback handling.

Express Server

The Express framework is used to create REST APIs.

Example server setup:

```
const express = require("express");
const app = express();
app.use(express.json());
app.get("/", (req, res) => {
res.send("My Personal Chef backend running");
});
```

User Authentication

The system provides APIs for user registration and login.

Example login API:

```
app.post("/login", async (req, res) => {
const { email, password } = req.body;
const user = await User.findOne({ email });
if (!user) {
return res.status(400).json({ error: "User not found" });
}
if (user.password !== password) {
return res.status(400).json({ error: "Wrong password" });
}
res.json({
message: "Login successful",
role: user.role,
name: user.name });
});
```

4.5 Database Implementation

The system uses **MongoDB** as the database. MongoDB stores data in the form of collections and documents.

The application uses **Mongoose**, an Object Data Modeling (ODM) library for MongoDB.

Recipe Collection

The Recipe collection stores information related to cooking recipes.

Fields include:

- title
- ingredients
- steps
- image
- cookingTime
- difficulty
- category
- servings
- youtubeUrl
- createdAt

User Collection

The User collection stores user account details.

Fields include:

- name
- email
- password
- role

- favorites

Feedback Collection

The Feedback collection stores feedback submitted by users for recipes.

Fields include:

- recipeId
- name
- email
- rating
- comment
- status
- createdAt

4.6 AI Integration (Gemini)

The system integrates **Google Gemini AI** to automatically extract recipe information from YouTube videos.

This feature allows administrators to paste a YouTube cooking video URL and automatically generate recipe details such as ingredients and steps.

The backend processes the video transcript and sends the text to the Gemini AI model.

Example prompt sent to Gemini:

Extract a cooking recipe from the given text and return the result in JSON format including title, ingredients, steps, cooking time, and difficulty level.

The response is then parsed and used to auto-fill the recipe form in the admin panel.

This feature improves productivity by reducing the need for manual recipe entry.

4.7 Summary

The coding and implementation phase converts the system design into a working application. The **My Personal Chef** system integrates modern web technologies such as React.js, Node.js, Express.js, and MongoDB to provide a dynamic recipe management platform.

The application also integrates **Google Gemini AI** to automatically extract recipe data from YouTube videos. This combination of technologies ensures an efficient, scalable, and user-friendly cooking assistant application.

CHAPTER – 5

OUTPUT SCREENS

5.OUTPUT SCREENS

CMD

To run the **My Personal Chef** application, both backend and frontend servers must be started.

Start backend server:

```
cd backend
node server.js
```

Start frontend server:

```
cd frontend
npm start
```

The frontend runs on **http://localhost:3000** and the backend runs on **http://localhost:5000**.

USER MODULE SCREENS

HOME PAGE

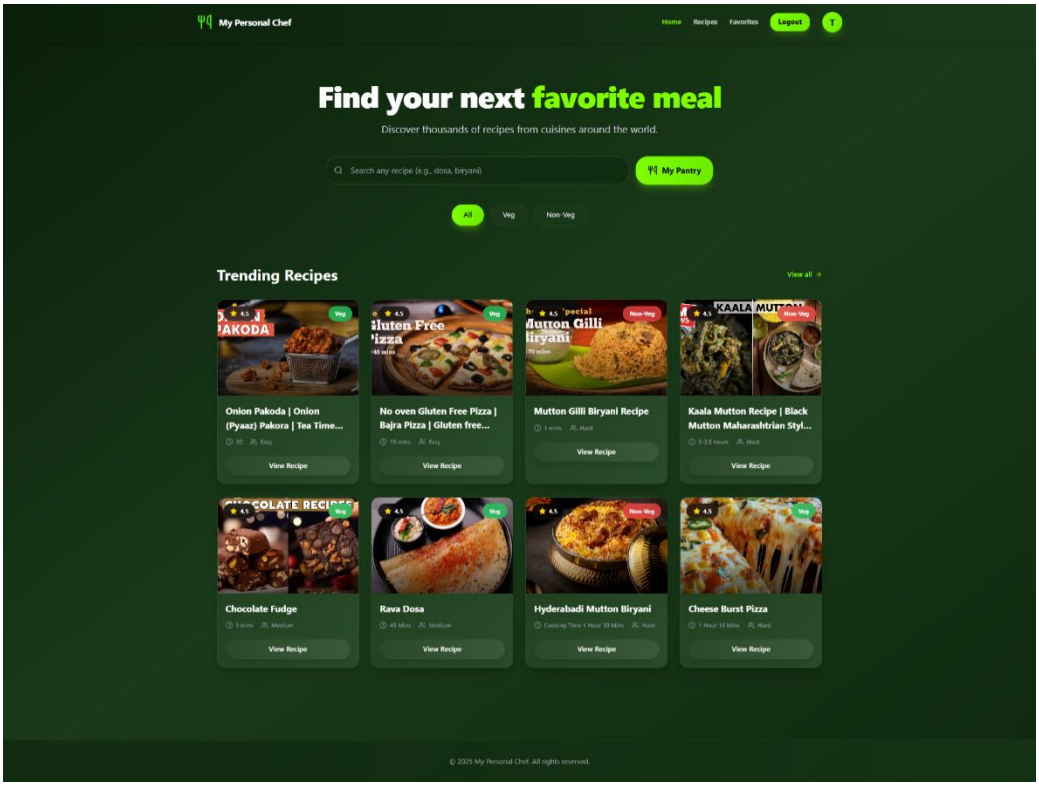


Fig 5.1 In the above screen is about the **Home page** of the My Personal Chef application. The home page allows users to explore recipes and navigate through different sections of the system.

LOGIN PAGE

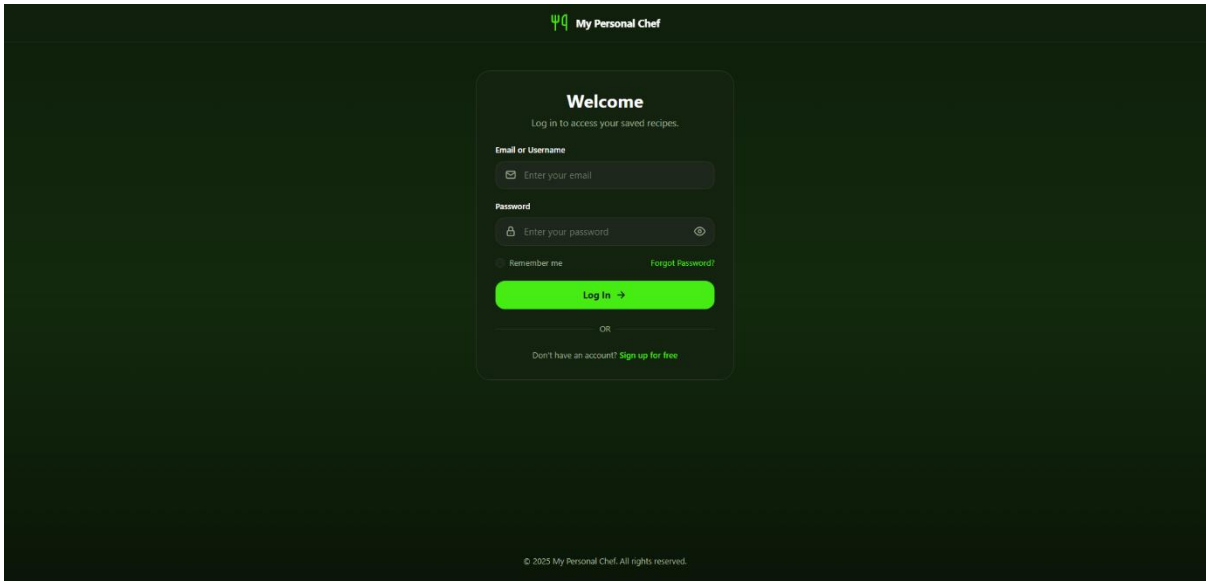


Fig 5.2 The above screen shows the **Login page**. Registered users can log in using their email and password to access personalized features of the system.

SIGNUP PAGE

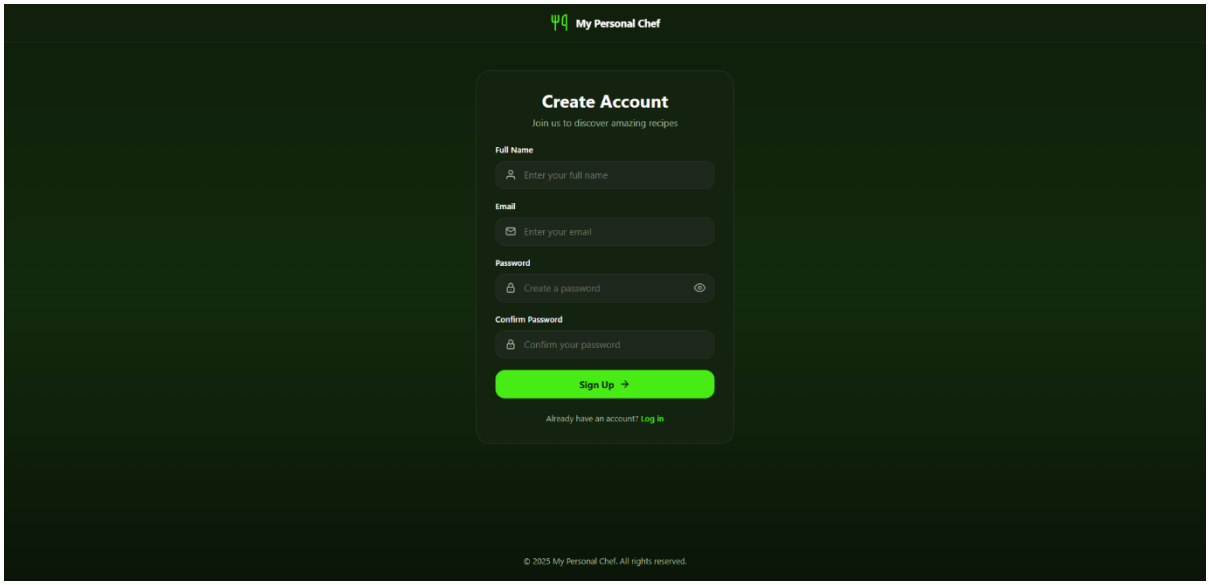


Fig 5.3 The above screen shows the **Signup page**. New users can create an account by providing their name, email address, and password.

RECIPES PAGE

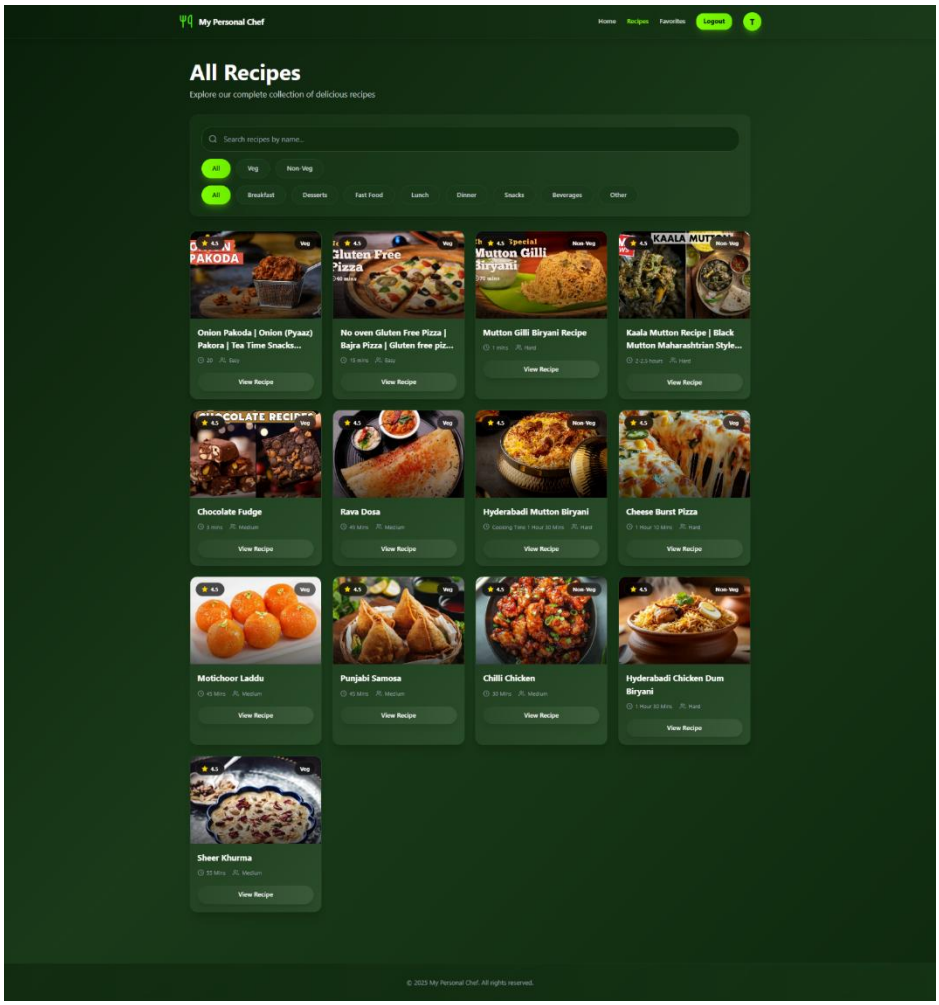


Fig 5.4 The above screen shows the **Recipes page** where users can browse available recipes. Users can filter recipes based on category and type.

RECIPE DETAIL PAGE

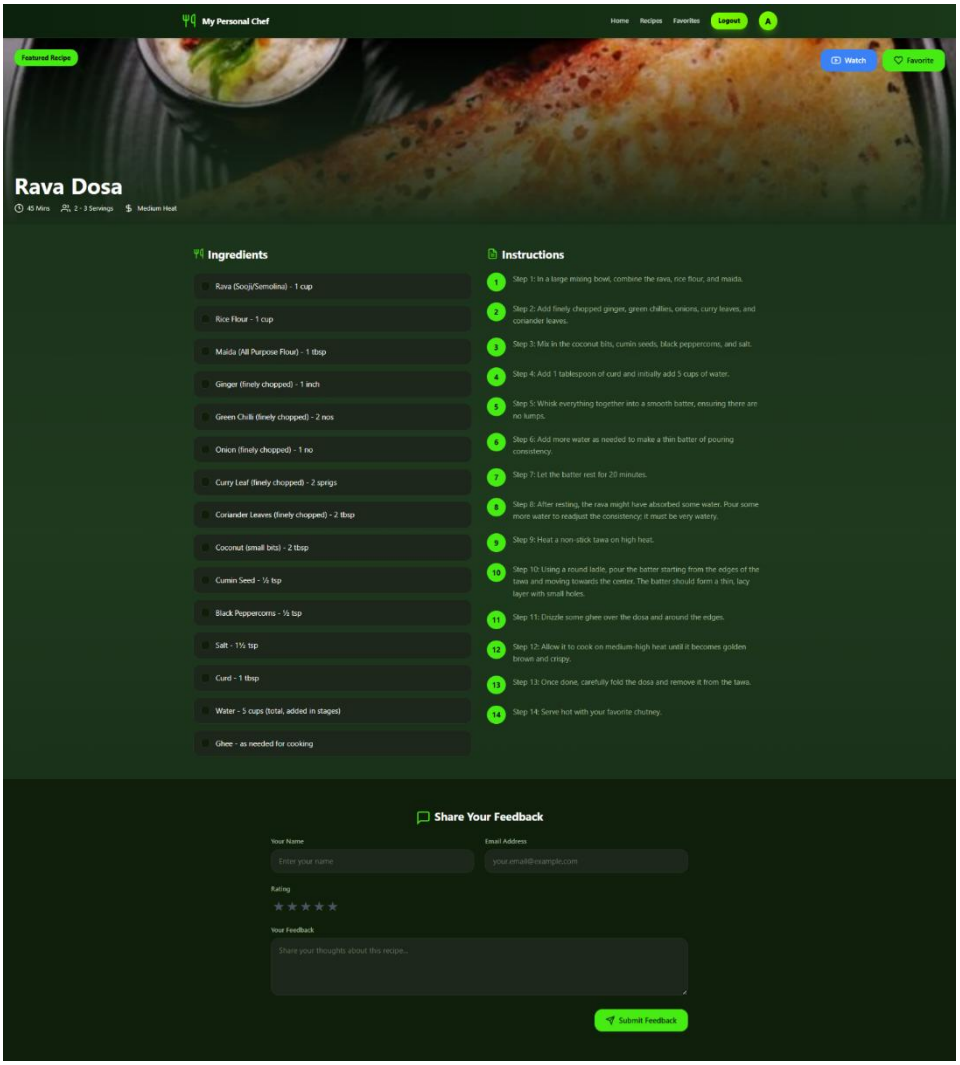


Fig 5.5 The above screen shows the **Recipe Detail page** which displays ingredients, cooking steps, cooking time, and difficulty level for the selected recipe.

PANTRY MATCH PAGE

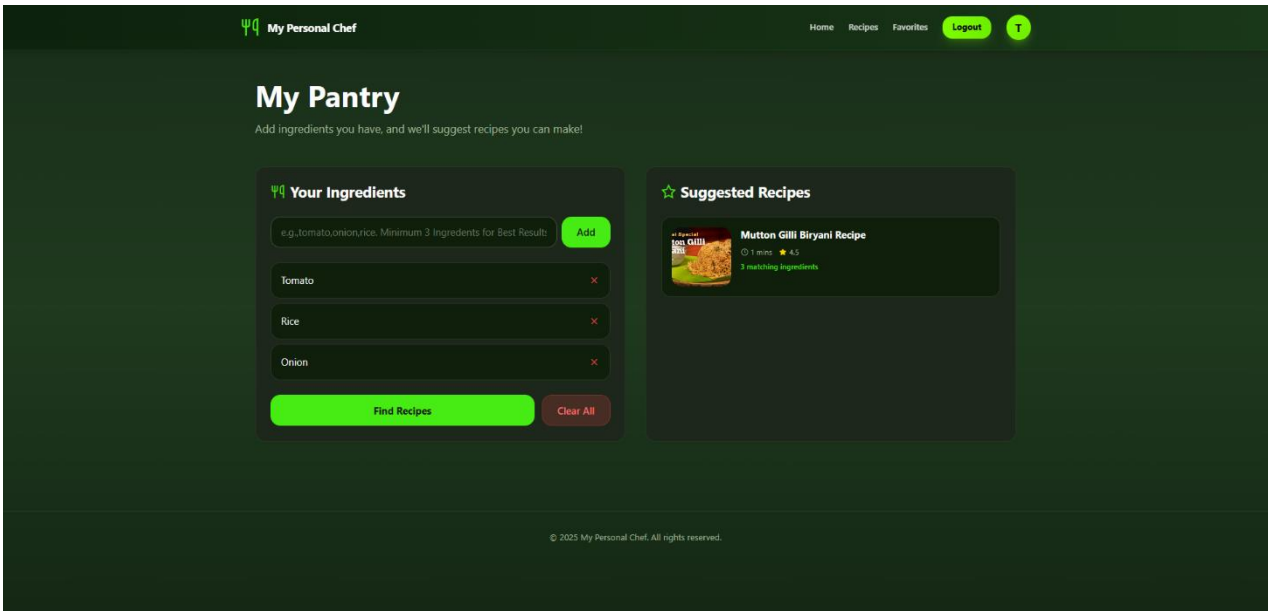


Fig 5.6 The above screen shows the **Pantry Suggestion page**. Users can enter ingredients they have and the system recommends recipes that match those ingredients.

FAVORITES PAGE

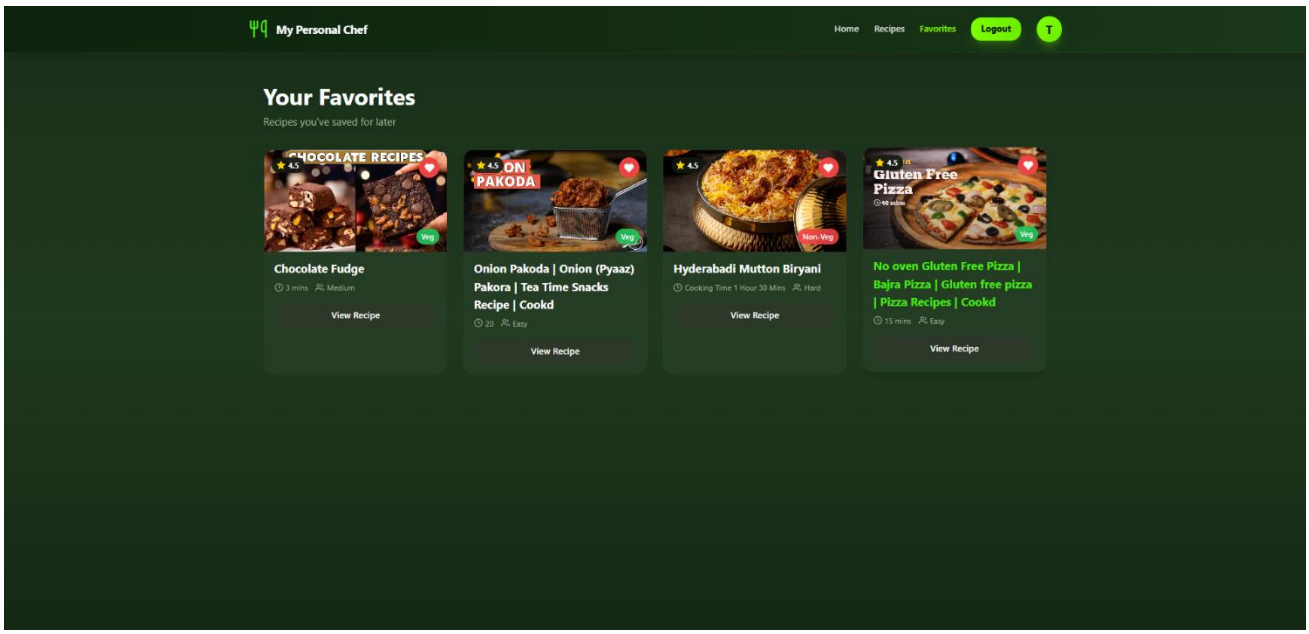


Fig 5.7 The above screen shows the **Favorites page** where users can view and manage their saved recipes.

ADMIN MODULE SCREENS

ADMIN ADD RECIPE PAGE

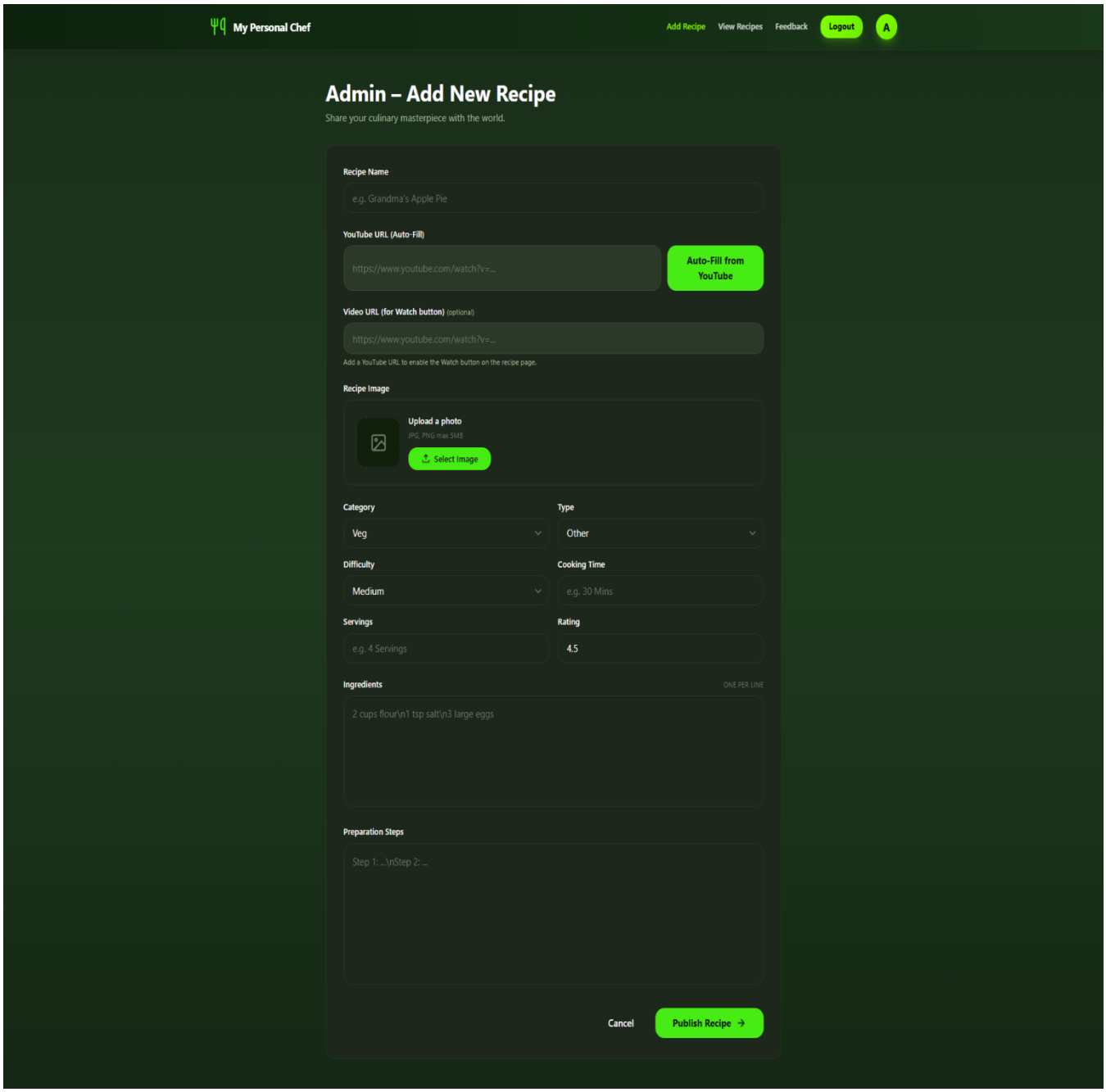


Fig 5.8 The above screen shows the **Admin Add Recipe page** where administrators can add new recipes to the system.

ADMIN RECIPE MANAGEMENT PAGE

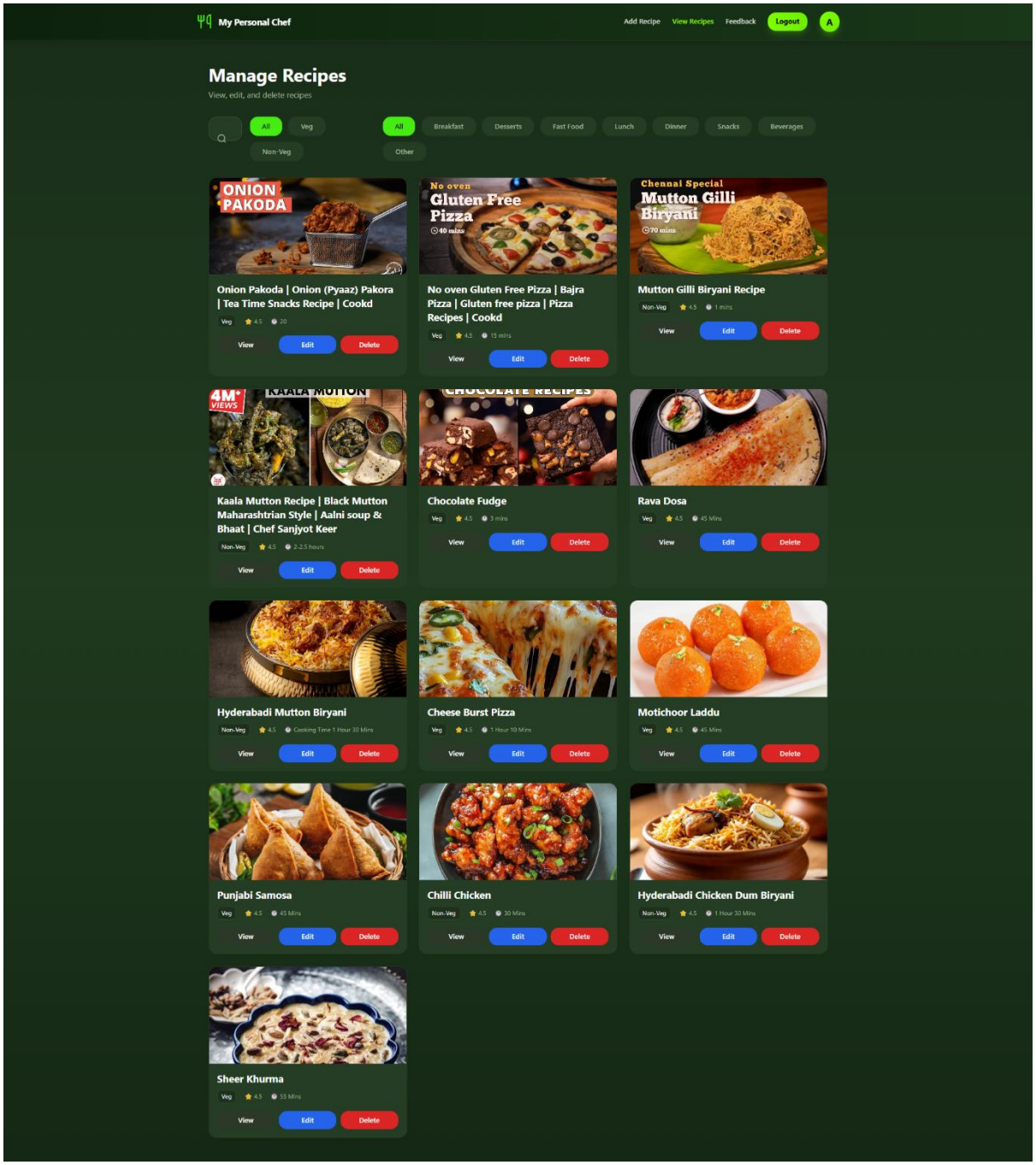


Fig 5.9 The above screen shows the **Admin View Recipes page** where administrators can edit or delete existing recipes.

ADMIN FEEDBACK MANAGEMENT PAGE

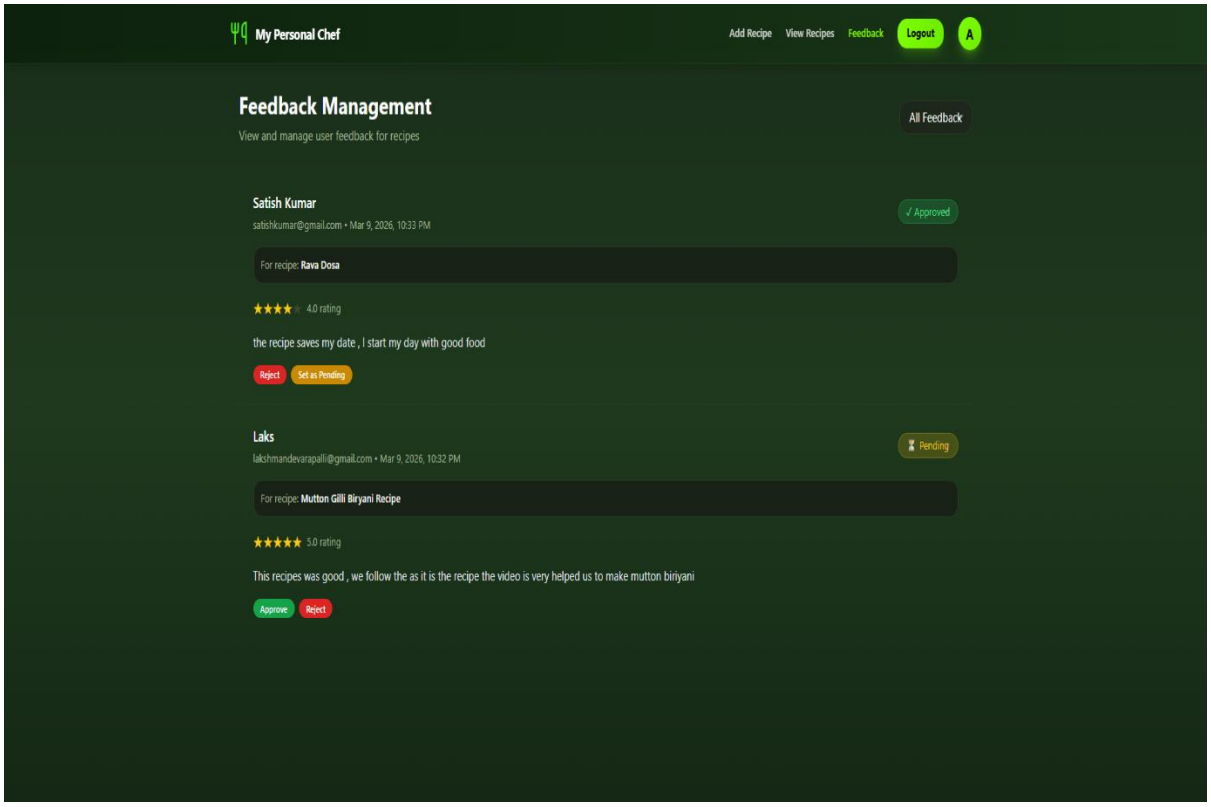


Fig 5.10 The above screen shows the **Admin Feedback page** where administrators can review and manage feedback submitted by users.

CHAPTER – 6

TESTING

6. TESTING

Testing in the **My Personal Chef Recipe Recommendation System** is an important phase that ensures the application functions correctly, efficiently, and reliably. The testing process verifies that different modules of the system operate as expected and interact smoothly with each other. The main objective of testing is to detect errors, improve system performance, and ensure that the application satisfies the functional requirements defined during system design.

At the basic level, **unit testing** is performed to verify the functionality of individual components such as user authentication, recipe retrieval, pantry matching logic, and feedback submission. These tests ensure that each module performs its specific function correctly without affecting other modules.

Integration testing is conducted to verify that the frontend and backend components of the system work together seamlessly. In this project, the React frontend communicates with the Node.js backend through REST APIs. Integration testing ensures that data is correctly transmitted between the frontend interface and the backend server and that the backend properly interacts with the MongoDB database.

Functional testing evaluates whether the system performs the tasks specified in the requirements. It verifies that users can successfully register, log in, browse recipes, view recipe details, add recipes to favorites, and receive recipe recommendations based on pantry ingredients.

Performance testing is also conducted to analyze how efficiently the system handles multiple requests and large data sets of recipes. This testing evaluates the response time of APIs, database queries, and the ability of the system to handle multiple users simultaneously.

By performing these testing techniques, the **My Personal Chef system ensures reliability, efficiency, and accuracy**, providing users with a smooth experience while browsing recipes and receiving personalized cooking suggestions.

6.1 TESTING TECHNOLOGIES

1. **Unit Testing**
    - **Node.js Testing Tools:** Used to test backend functions and API logic.
    - **JavaScript Testing:** Used to validate frontend component functionality.
2. **Integration Testing**
    - **React + Express Integration:** Ensures that the React frontend successfully communicates with backend APIs.
    - **MongoDB Integration Testing:** Verifies correct interaction between backend services and the database.
3. **API Testing**
    - **Postman:** Used to test REST APIs such as login, recipe retrieval, pantry matching, and feedback submission.
    - **Browser Developer Tools:** Used to inspect API responses and debug network requests.
4. **Database Testing**
    - **MongoDB Compass:** Used to verify stored data, collections, and database operations.
5. **User Interface Testing**
    - **Google Chrome Browser:** Used to test the frontend interface and verify UI responsiveness.

6.2 TEST CASES

Test Case 1: User Registration

Input: Valid user details (name, email, password)
Expected Output: User account created successfully.

Test Steps:

- Enter user details in the signup form.
- Click the register button.
- Verify that the user account is created and stored in the database.

Test Case 2: User Login

Input: Registered email and password
Expected Output: User successfully logged into the system.

Test Steps:

- Enter login credentials.
- Click the login button.
- Verify that the user is redirected to the home page.

Test Case 3: Recipe Browsing

Input: Open the recipes page
Expected Output: List of recipes displayed from the database.

Test Steps:

- Navigate to the recipes page.
- Verify that recipe cards are displayed correctly.

Test Case 4: Recipe Detail View

Input: Select a recipe from the recipes page

Expected Output: Detailed information about the recipe is displayed.

Test Steps:

- Click on a recipe card.
- Verify that ingredients, steps, and cooking time are shown.

Test Case 5: Pantry Ingredient Matching

Input: Enter ingredients such as tomato, onion, rice

Expected Output: Recipes matching the entered ingredients are displayed.

Test Steps:

- Enter ingredients in the pantry page.
- Click the search button.
- Verify that matching recipes are suggested.

Test Case 6: Add Recipe to Favorites

Input: Click the favorite button on a recipe

Expected Output: Recipe is added to the favorites list.

Test Steps:

- Select a recipe.
- Click the favorite icon.
- Verify that the recipe appears in the favorites page.

Test Case 7: Admin Add Recipe

Input: Enter recipe details in admin panel

Expected Output: Recipe added successfully to the database.

Test Steps:

- Login as admin.
- Enter recipe details.
- Click publish recipe.
- Verify that the recipe appears in the recipes page.

Test Case 8: Feedback Submission

Input: Submit feedback for a recipe

Expected Output: Feedback stored in the database.

Test Steps:

- Enter feedback details.
- Submit the form.
- Verify that feedback appears in the admin feedback panel.

CHAPTER – 7

CONCLUSION

7.CONCLUSION

The **My Personal Chef** application was successfully designed and implemented using modern web technologies such as **React.js, Node.js, Express.js, and MongoDB**. The main objective of this project was to develop a user-friendly system that helps users easily discover recipes, manage favorite recipes, and receive intelligent recipe suggestions based on ingredients available at home.

The system provides an interactive interface where users can browse recipes, view detailed cooking instructions, and save recipes for later use. One of the key features of the system is the **Pantry Matching feature**, which allows users to enter ingredients they already have and receive recipe suggestions based on matching ingredients. This feature helps users reduce food waste and quickly decide what they can cook with available ingredients.

The project also includes an **administrative module** that allows administrators to manage recipe data efficiently. Administrators can add new recipes, edit existing recipes, delete recipes, and review user feedback. This ensures that the recipe database remains organized and continuously updated.

Another important feature of the system is the integration of **Artificial Intelligence using Google Gemini AI**. This functionality allows administrators to automatically extract recipe information from YouTube cooking videos. The AI analyzes the video transcript or description and generates structured recipe data such as ingredients, cooking steps, cooking time, and difficulty level. This significantly reduces manual effort and improves productivity when adding recipes to the system.

The system was tested using various testing methods including **unit testing, integration testing, functional testing, and performance testing** to ensure reliability and proper functioning of all modules. The results confirmed that the system performs as expected and provides accurate and efficient recipe recommendations.

Overall, the **My Personal Chef** system demonstrates the practical implementation of a full-stack web application using the MERN stack combined with AI-based features. The application successfully achieves its goal of simplifying recipe discovery and providing personalized cooking assistance to users.

CHAPTER – 8

FUTURE ENHANCEMENTS

8.FUTURE ENHANCEMENTS

Future enhancements for the **My Personal Chef** system can focus on improving functionality, personalization, and user experience. By integrating advanced technologies, the system can become a more intelligent and efficient cooking assistant.

One possible improvement is the use of **machine learning algorithms** to provide personalized recipe recommendations based on user preferences, previous searches, and favorite recipes. This would help users discover recipes that better match their interests.

Another enhancement is the inclusion of **nutritional information and diet-based filtering**. The system could display calorie counts and other nutritional values for each recipe, helping users follow healthier diets such as vegetarian, vegan, or low-calorie meal plans.

The system can also be extended with a **mobile application version**, allowing users to access recipes, manage their pantry ingredients, and receive recommendations easily from their smartphones.

Additionally, **community features** could be introduced where users can share recipes, rate dishes, and provide reviews. Cloud deployment and improved AI-based recipe extraction from various sources such as cooking websites and blogs could further enhance the system.

These improvements will make the **My Personal Chef** application more powerful, interactive, and user-friendly in the future.

CHAPTER - 9

BIBLIOGRAPHY

9.BIBLIOGRAPHY

Here are some references used for developing the **My Personal Chef Recipe Recommendation System**.

Research Papers

1. "Food Recommendation Systems: A Survey" by L. Vargas et al. (2021)
2. "Recipe Recommendation Using Ingredient Networks" by Y. Ahn et al. (2011)
3. "Personalized Food Recommendation Systems Using Machine Learning" by S. Trattner et al. (2017)
4. "Deep Learning Approaches for Food Recognition and Recommendation" by K. Okamoto et al. (2018)
5. "Intelligent Recipe Recommendation Systems Based on User Preferences" by J. Freyne et al. (2010)

Books

6. "Learning React: Modern Patterns for Developing React Apps" by Alex Banks and Eve Porcello (2020)
7. "Node.js Design Patterns" by Mario Casciaro and Luciano Mammino (2020)
8. "MongoDB: The Definitive Guide" by Shannon Bradshaw et al. (2019)
9. "JavaScript: The Definitive Guide" by David Flanagan (2020)
10. "Artificial Intelligence: A Modern Approach" by Stuart Russell and Peter Norvig (2021)

Journals

11. Journal of Artificial Intelligence Research (JAIR)
12. International Journal of Data Science and Analytics
13. Journal of Web Engineering
14. Journal of Machine Learning Research (JMLR)
15. International Journal of Computer Science and Applications

Conferences

16. International Conference on Artificial Intelligence and Data Science (ICAIDS)
17. International Conference on Machine Learning Applications (ICMLA)
18. International Conference on Data Mining and Knowledge Discovery (ICDMKD)
19. Conference on Web Engineering and Modern Applications (ICWE)
20. International Conference on Software Engineering and Applications (ICSEA)

CHAPTER – 10

JOURNAL PUBLICATION

10. JOURNAL PUBLICATION